

Chapter 6

Introduction to Computer Architecture



Objectives: After Completing this Chapter, you Should be Able to:

- List the components of a microcomputer.
- List the components of CPU.
- Distinguish CPU technologies.
- Learn architecture of multicore processor.
- Compare the RISC processor with the CISC processor.
- Explain the difference between the von Neumann and the Harvard architecture.
- Distinguish between the 32-bit processor and the 64-bit processor.
- Explain the instruction execution steps.
- Show advantage of the instruction pipelining.
- Distinguish different types of the microcomputer buses.
- Explain operation of the USB bus.

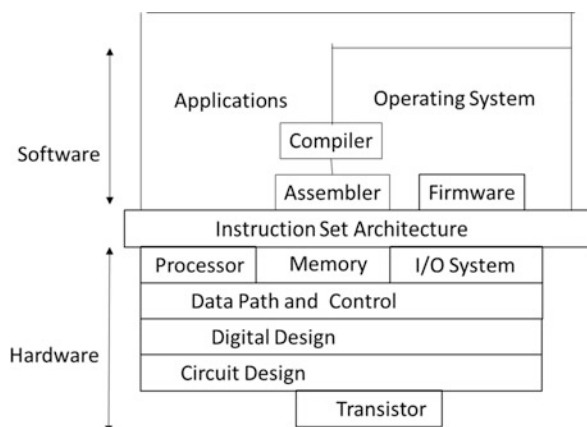
6.1 Introduction

Just as the architecture of a building defines its overall design and functions, so computer architecture defines the design and functionality of a computer system. The components of a microcomputer are designed to interact with one another, and this interaction plays an important role in the overall system operation.

6.1.1 Abstract Representation of Computer Architecture

A general view of the layered architecture of computer is shown in Fig. 6.1. In this figure, it shows that the Instruction Set architecture is located between hardware and software as it acts as the interface layer.

Fig. 6.1 Layered architecture of a computer



Compiler: The function of a **compiler** is to convert a high-level language (HLL) to assembly language, and the assembler then converts assembly language to machine code (binary). Some of the most popular HLLs are C, C++, Java, and Python.

Firmware: Firmware is a software program or set of instructions programmed on a hardware device. It provides the necessary instructions for how the device communicates with the other computer hardware devices such as the BIOS, device manager, and device driver.

Instruction Set Architecture: The instruction set are the set of different commands that are supported by the hardware. It is basically the interface between the hardware and software layers.

Applications: Applications are programs run by users like text editors and web browsers.

The hardware layer consists of the CPU, memory, and I/O devices that make up the computer.

Operating System: Operating systems manage computer hardware resources such as input/output operations, managing memory, and scheduling processes for execution.

6.2 Components of a Microcomputer

A standard microcomputer consists of a microprocessor (CPU), buses, memory, serial input/output, programmable I/O interrupt, and direct memory access DMA. Figure 6.2 shows components of microcomputer.

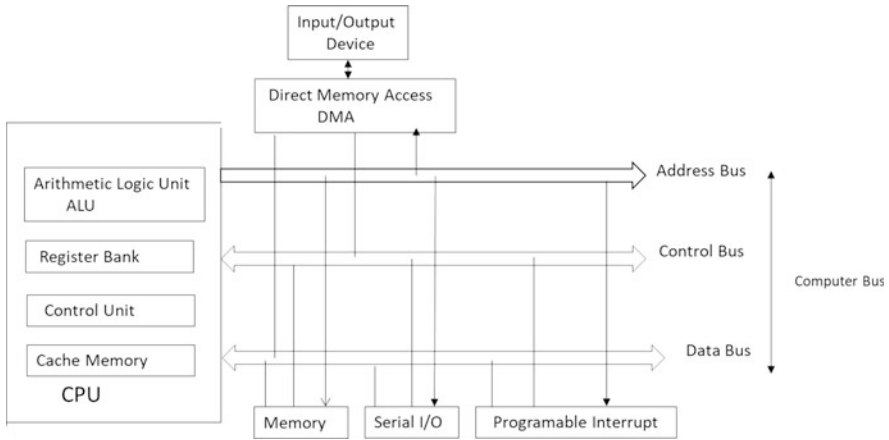


Fig. 6.2 Components of a microcomputer

6.2.1 Central Processing Unit (CPU)

The *central processing unit (CPU)* is the “brain” of the computer and is responsible for accepting data from input devices, processing the data into information, and transferring the information to memory and output devices. The CPU is organized into the following three major sections:

1. Arithmetic logic unit (ALU)
2. Control unit
3. Registers

The function of the *arithmetic logic unit (ALU)* is to perform arithmetic operations such as addition, subtraction, division, and multiplication and logic operations such as AND, OR, and NOT.

The function of the *control unit* is to control input/output devices, generate control signals to the other components of the computer such as read and write signals, and perform instruction execution. Information is moved from memory to the registers; the registers then pass the information to the ALU for logic and arithmetic operations.

6.2.1.1 Register Bank

Register is the fastest memory in a computer which holds information.

6.2.2 CPU Buses

When more than one wire carries the same type of information, it is called a bus. The most common buses inside a microcomputer are the address bus, the data bus, and the control bus.

6.2.2.1 Address Bus

The address bus defines the number of addressable locations in a memory IC by using the 2^n formula, where n represents the number of address lines. If the address bus is made up of three lines, then there are $2^3 = 8$ addressable memory locations, as shown in Fig. 6.3. The size of the address bus directly determines the maximum numbers of memory locations that can be accessed by the CPU. For example, a CPU with 32 address bus can have 2^{32} addressable memory locations.

6.2.2.2 Data Bus

The data bus is used to carry data to and from the memory. In Fig. 6.3, each location can hold only 8 bits. The size of a memory IC is represented by $2^n \times m$ where n is the number of address lines and m is the size of each location, usually each memory location holds one byte. In Fig. 6.3, where $n = 3$ and $m = 8$, the size of the memory is

$$2^3 * 8 = 64 \text{ bits.}$$

The size of data bus plays an important factor on CPU performance, current CPU's data bus is 32 bits or 64 bits, and a CPU with 32-bit data bus means it can read or write 32 bits of data in and from memory.

Early generation of CPU contains 8-bit data bus, and each memory location holds one byte, for reading word “book” as shown in Fig. 6.4. The CPU requires to access memory four times.

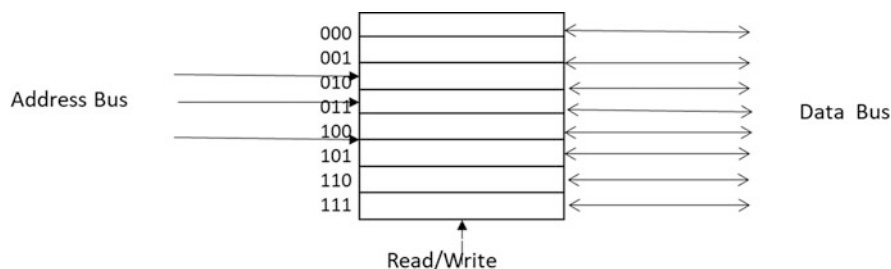


Fig. 6.3 A memory with three address lines and four data lines

Fig. 6.4 CPU with 8 bit Data Bus



By increasing data bus from 8 bits to 32 bits, then CPU can access memory and read entire word “book.” Most CPUs offer instruction to read 1 byte, 2 bytes, or 4 bytes from memory. Most CPUs can read from memory or write in memory in sizes of one byte, two bytes, and four bytes at a time.

6.2.2.3 Control Bus

The control bus carries control signals from the control unit to the computer components in order to control the operation of each component. In addition, the control unit receives control signals from computer components. Some of the control signals are as follows:

- Read signal:* The read line is set to high to read from memory location or input/output (I/O) devices
- Write signal:* The write line is used to write data into the memory
- Interrupt:* Indicates an interrupt request
- Bus request:* The device is requesting to use the computer bus
- Bus Grant:* Gives permission to the requesting device to use the computer bus
- I/O Read and Write:* I/O read and write are used to read from or write to I/O devices

6.2.3 Memory

There are four types of memory used in a computer: registers, cache memory, main memory, and secondary memory.

Register: Registers are located inside CPU and hold.

Cache memory: Cache memory is part of the CPU and it is the fastest type of memory. Cache is made of SRAM (Static Random Access Memory).

Main Memory: Main memory is a type of DRAM (Dynamic Random Access Memory) and is volatile and fast.

Secondary Memory: Secondary memory is also called disk or secondary storage, and includes mass storage devices like hard disks or solid-state drives (SSD). This type of memory is slow and less expensive when compared with DRAM.

6.2.4 *Serial Input/Output*

To interface with other devices, computers need to use different kinds of serial input and output interfaces. The most popular devices that utilize input/output interfaces include USB devices and PCI Express.

6.2.5 *Direct Memory Access (DMA)*

Direct memory access (DMA) allows for the transfer of blocks of data from memory to an I/O device or vice versa. Without DMA, the CPU reads data from memory and writes it to an I/O device. Transferring blocks of data from memory to an I/O device requires the CPU to do one read and one write for each operation. This method of data transfer takes a lot of time. The function of DMA is to transfer data from memory to an I/O device directly, without using the CPU, so that the CPU is free to perform other functions.

The DMA performs the following functions in order to use the computer bus:

- The DMA sends a request signal to the CPU.
- The CPU responds to the DMA with a grant request, permitting the DMA to use the bus.
- The DMA controls the bus and the I/O device is able to read or write directly to or from memory.
- The DMA is able to load a file off an external disk into main memory when large blocks of data need to be transferred to a sequential range of memory. DMA is much faster and more efficient than a CPU.

6.2.6 *Programmable I/O Interrupt*

When multiple I/O devices such as external drives, hard disks, printers, monitors, and modems are connected to a computer as shown in Fig.6.5, a mechanism is necessary to synchronize all device requests. The function of a programmable interrupt is to check the status of each device and inform the CPU of the status of each; for example, the printer is not ready, a disk is write protected, this is an unformatted disk, and there is a missing connection to a modem. Each device

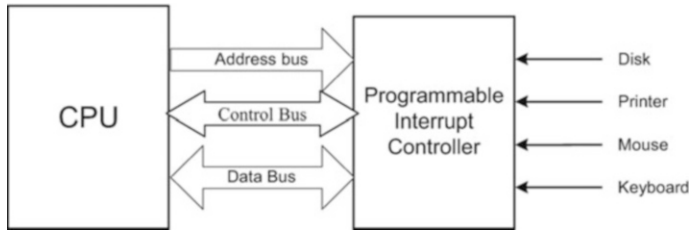


Fig. 6.5 Programmable interrupt controller

sends a signal to the programmable I/O interrupt controller in order to update its status. Figure 6.5 shows the programmable I/O interrupt controller.

6.2.7 32-Bit Versus 64-Bit CPU

The size of register plays an important role in the performance of CPU. A 32-bit processor can perform operations on 32-bit data; therefore, the size of registers is 32 bits and ALU also performs 32-bit operations. A 64-bit CPU performs operation in 64-bit data; therefore, it contains 64-bit register and 64-bit ALU.

Most desktop and server computers are using AMD and Intel processors; they might use 32 bits or 64 bits. Intel and AMD processor use the same architecture; this means a program in computer with Intel processor can run on a computer with AMD processor.

6.3 CPU Technology

There are two types of technology used for designing CPU and they are called CISC and RIS.

6.3.1 CISC (Complex Instruction Set Computer)

In 1978, Intel developed the 8086 microprocessor chip. The 8086 was designed to process a 16-bit data word; it had no instruction for floating point operations. At the present time, the Pentium processes 32-bit and 64-bit words, and it can process floating point instructions. Intel designed the Pentium processor in such a way that it can execute programs written for earlier 80×86 processors.

The characteristics of 80×86 are called complex instruction set computers (CISC), which include instructions for earlier Intel processors. Another CISC

processor is VAX 11/780, which can execute programs for the PDP-11 computer. The CISC processor contains many instructions with different addressing modes, for example, the VAX 11/780 has more than 300 instructions with 16 different address modes.

The major characteristics of CISC processor are as follows:

1. A large number of instructions.
2. Many addressing modes.
3. Variable length of instructions.
4. Most instruction can manipulate operands in the memory.
5. Control unit is microprogrammed.

6.3.2 RISC

Until the mid-1990s, computer manufacturers were designing complex CPUs with large sets of instructions. At that time, a number of computer manufacturers decided to design CPUs capable of executing only a very limited set of instructions.

One advantage of reduced instruction set computer is that they can execute their instructions very fast because the instructions are simple. In addition, the RISC chip requires fewer transistors than the CISC chip. Some of the RISC processors are the PowerPC, MIPS processor, IBM RISC System/6000, ARM, and SPARC.

The major characteristics of RISC processors are as follows:

1. Require few instructions.
2. All instructions are the same length (they can be easily decoded).
3. Most instructions are executed in one machine clock cycle.
4. Control unit is hardwired.
5. Few address modes.
6. A large number of registers.

RISC processor uses hardware and CISC processor microprogram for control unit, and the control unit with hardware uses less space in a CPU; therefore, the designer of CPU can add more registers to RISC processor compared with CISC.

The advantage of CISC processor is that designer can add new instruction without changing the architecture of the processor. Table 6.1 shows the comparison of CISC and RISC.

Table 6.1 Comparison of RISC and CISC processor

CISC	RISC
Variable instruction length	Fixed instruction length
Variable opcode length	Fixed opcode length
Memory operands	Load/store instructions
Example: Pentium	ARM, MIPS

6.4 CPU Architecture

There are two types of CPU architecture and they are *von Neumann* and *Harvard architecture*.

6.4.1 Von Neumann Architecture

It is a program consisting of code (instructions) and data. Figure 6.6 shows a block diagram of the von Neumann architecture. Von Neumann uses the data bus to transfer data and instructions from the memory to the CPU.

6.4.2 Harvard Architecture

Harvard architecture uses separate buses for instructions and data as shown in Fig. 6.7. The instruction address bus and instruction bus are used for reading instructions from memory. The address bus and data bus are used for writing and reading data to and from memory.

Fig. 6.6 Von Neumann architecture

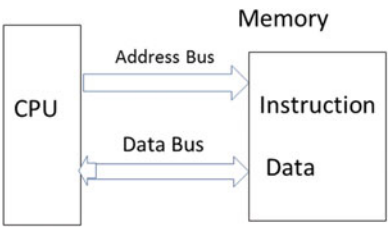
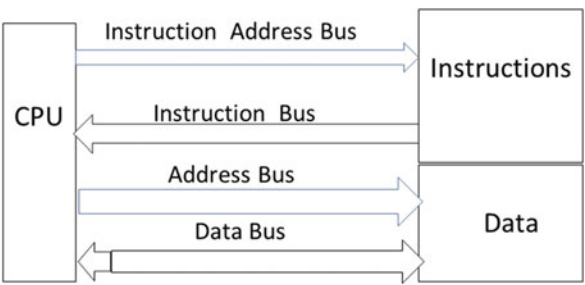


Fig. 6.7 Harvard architecture



6.5 Intel Microprocessor Family

Intel designs and manufactures microprocessors for microcomputers. Each processor has a number or name, which is used by the computer designer to access the information provided by the manufacturer of the processor.

Intel microprocessor IC numbers and names are 8088, 80286, 80386, 80486, Pentium, Pentium II, Pentium III, and Pentium IV which they called IA-86 (Intel architecture-86). Recently, Intel and HP developed a new processor called Itanium which is a 64-bit processor. The following is a list of the characteristics of Intel microprocessor (Table 6.2):

Most workstations or laptops use CPU which is manufactured by Intel and AMD Corporations. Intel processor is classified by the IA-16 (Intel architecture 16-bit processor), IA-32, and IA-64.

6.5.1 Upward Compatibility

Intel architecture is upward compatible meaning a program written for IA-16 processor can run on IA-32.

Figure 6.8 shows general register IA-16, where AH, AL, BH, BL, CH, CL, DH, and DL are 8-bit registers and AX, BX, CX, and DX are 16-bit register. The AX, BX, CX, and DX are combinations of two registers.

Table 6.2 Characteristics of Intel microprocessor

	80486dx	Pentium	Pentium Pro	Pentium Pro II	Pentium II
Register size	32 bits	32 bits	32 bits	32/64 bits	32/64 bits
Data bus size	32 bits	64 bits	64 bits	64 bits	64 bits
Address size	32 bits	32 bits	32 bits	32 bits	32 bits
Max memory	4 GB	4 GB	4 GB	4 GB	4 GB
Clock speed	25, 33 MHz	60, 166 MHz	150, 200 MHz	233, 340, 400 MHz	450, 500 MHz
Math processor	Built-in	Built-in	Built-in	Built-in	Built-in
L1 cache	8 KB, 16 KB	8 KB instruction 8 KB data	8 KB instruction 8 KB data	16 KB instruction 16 KB data	16 KB instruction 16 KB data
L2 cache	No	No	256 KB or 512 KB	512 KB	512 KB
MMX technology	No	No	Yes	Yes	Yes

L1 cache is the cache memory built inside the microprocessor.

L2 cache is not part of microprocessor; it is in a separate IC

Fig. 6.8 IA-16 registers

AX	AH	AL
BX	BH	BL
CX	CH	CL
DX	DH	DL

		16 bits		
EAX		AH	AL	AX
EBX		BH	BL	BX
ECX		CH	CL	CX
EDX		DH	DL	DX
	32 bits			

Fig. 6.9 Intel IA-32-bit registers

Figure 6.9 shows IA-32 general registers where EAX, EBX, ECX, and EDX are general registers, and also Fig. 6.9 also contains IA-16-bit registers; therefore, a program was written for IA-16 can be executed by IA-32.

6.6 Multicore Processors

A *multicore processor* is an integrated circuit (IC) with two or more independent CPU which is called core, and they are executing multiple instructions simultaneously in order to increase performance. A quad-core *processor* is a *chip* with four independent units called cores that read and execute instructions such as add, move data, and branch. Figure 6.10 shows a block diagram of quad-core processors which are all sharing a memory. The following are some of the multicore processors:

- Two cores (dual-core CPUs) such as AMD Phenom II X2 and Intel Core Duo
- Three cores (tri-core CPUs) such as AMD Phenom II X3
- Four cores (quad-core CPUs) such as AMD Phenom II X4, Intel’s i5 and i7 processors
- Six cores (hexa-core CPUs) such as AMD Phenom II X6 and Intel Core i7 Extreme Edition 980X

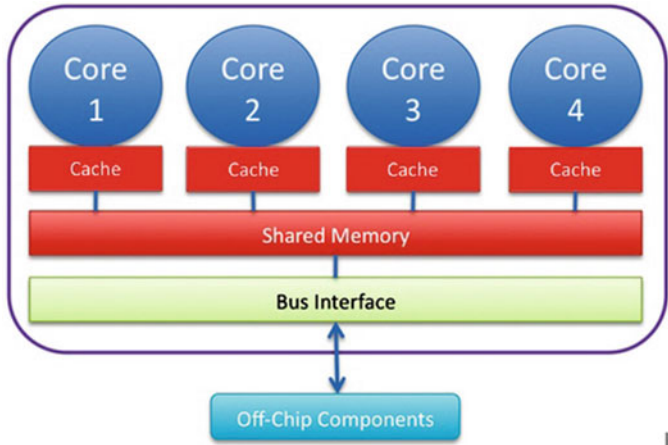


Fig. 6.10 Multicore processor architecture

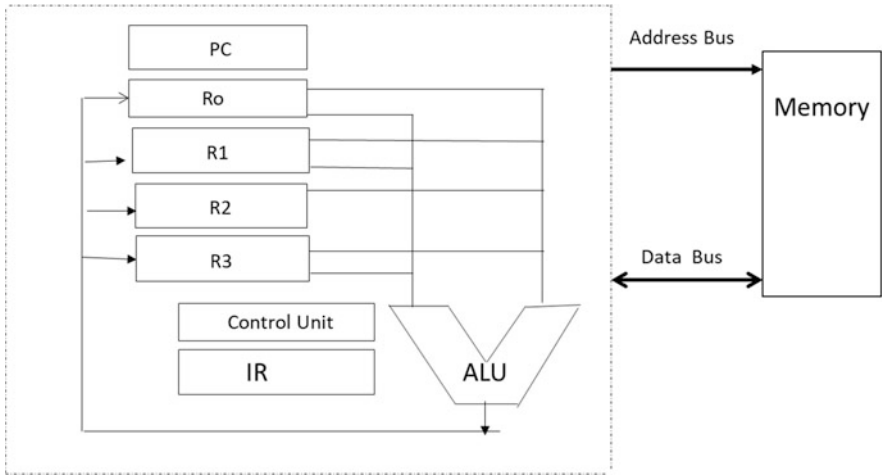


Fig. 6.11 Internal components of a CPU

Eight cores (octa-core CPUs) such as Intel Core i7 5960X Extreme Edition and AMD FX-8350

Ten cores (deca-core CPUs) such as Intel Xeon E7-2850

6.7 CPU Instruction Execution Steps

Figure 6.11 shows the internal components of a CPU, which consists of data registers R0, through R3, PC (Program Counter), IR (Instruction Register), ALU, and Control Unit.

The PC (Program Counter) holds the address of the next instruction to be executed in a program, and the IR (Instruction Register) holds that instruction.

In general, the CPU performs the following steps to execute one instruction:

- 1. **Fetch instruction (IF):** The Fetch instruction is when the operation system uploads a program into the memory of the computer, then stores the starting address of the program on the PC. Then the control unit places the contents of the PC onto the address bus, reads the instruction from memory, and stores it in the IR. The PC will then increment for the next instruction to be fetched. In summary, the moving of an instruction from memory to the IR is called the Fetch instruction.
- 2. **Decode instruction (D):** Now that the IR holds the instruction (such as ADD R3, R2,R1, or AND R3,R1,R2), the control unit will decode the type of instruction stored and move the specified register contents into inputs of the ALU.
- 3. **Execute instruction (E):** With ALU registers loaded with the correct data, the instruction will be executed using the specified operation.
- 4. **Write results (R):** After execution, the results of the calculation (output of ALU) are stored into a register or memory.

6.7.1 Pipelining

Pipelining will increase the performance of CPU meaning executing more instructions in less time.

Figure 6.12 shows execution of four instructions without pipelining; in this figure, CPU executes one instruction at a time, and each stage takes T second; then total execution time is 16 T.

Figure 6.13 shows execution of four instructions using pipeline; CPU fetches I1 instruction and moves it to decode unit; CPU while decoding I1 will fetch I2 instruction and this process will continue.

As shown in Fig. 6.13, it takes 7 T to complete the execution of four instructions. Figure 6.13 shows that at time T4 CPU writes the results of execution I1 into memory and at the same time fetches instruction I4 from memory, but it is



Fig. 6.12 Execution of instruction without pipeline

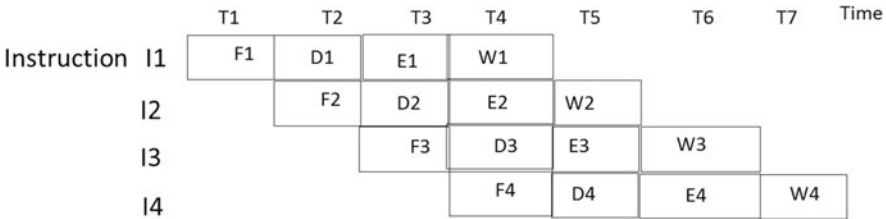


Fig. 6.13 Execution of instruction using pipeline

impossible to read and write at the same time into or from memory; therefore, having two separate caches (instruction cache and data cache) will overcome this conflict, and this type of architecture is called Harvard architecture.

6.8 Disk Controller

The disk controller moves the disk drive head, reads, and/or writes data. The most popular disk controllers are IDE (integrated disk electronics) and **SATA (Serial Advanced Technology Attachment)** that is a [computer bus](#) interface that connects [host bus adapters](#) to [mass storage devices](#) such as [hard disk](#).

6.9 Microcomputer Bus

There are currently a number of different computer buses on the market that are designed for microcomputers. Some of the computer BUS are ISA, MCA, EISA, VESA PCI, FireWire, USB, and PCI Express. Universal serial bus (USB) and PC Express are covered in more detail because they are more advanced than other buses.

6.9.1 ISA Bus

The *industry standard architecture (ISA) bus* was introduced by IBM for the IBM PC using an 8088 microprocessor. The ISA bus has an 8-bit data bus and 20 address lines at a clock speed of 8 MHz. The PC AT type uses the 80,286 processor which has a 16-bit data bus and 24-bit address lines and is compatible with the PC.

6.9.2 *Microchannel Architecture Bus*

The *microchannel architecture (MCA) bus* was introduced by IBM in 1987 for its PS/2 microcomputer. The MCA bus is a 32-bit bus that can transfer four bytes of data at a time and runs at a 10 MHz clock speed. It also supports 16-bit data transfer and has 32-bit address lines. Microchannel architecture was so expensive, the non-IBM vendors developed a comparable but less expensive solution called the EISA bus.

6.9.3 *EISA Bus*

The *extended ISA (EISA) bus* is a 32-bit bus that also supports 8- and 16-bit data transfer bus architectures. EISA runs at 8-MHz clock speeds and has 32-bit address lines.

6.9.4 *VESA Bus*

The *video electronics standard association (VESA) bus*, which is also called a video local bus (VL-BUS), is a standard interface between the computer and its expansion. As applications became more graphically intensive, the VESA bus was introduced to maximize throughput of video graphics memory. The VESA bus provides fast data flow between stations and can transfer up to 132 Mbps.

6.9.5 *PCI Bus*

The *peripheral component interconnect (PCI) bus* was developed by Intel Corporation. PCI bus technology includes a 32-/64-bit bus that runs at a 33/66 MHz clock speed. PCI offers many advantages for connections to hubs, routers, and network interface cards (NIC). In particular, PCI provides more bandwidth: up to 1 gigabit per second as needed by these hardware components.

The PCI bus was designed to improve the bandwidth and decrease latency in computer systems. Current versions of the PCI bus support data rates of 1056 Mbps and can be upgraded to 4224 Mbps. The PCI bus can support up to 16 slots or devices in the motherboard. Most suppliers of ATM (asynchronous transfer mode) and 100BaseT NICs offer a PCI interface for their products. The PCI bus can be expanded to support a 64-bit data bus. Table 6.3 compares different bus architectures showing characteristics of ISA, EISA, MCA, VESA, and PCI buses. Figure 6.14 shows the PCI bus.

Table 6.3 Characteristics of various buses

Bus type	ISA	EISA	MCA	VESA	PCI	PCI-64
Speed (MHz)	8	8.3	10	33	33	64
Data bus bandwidth (bits)	16	32	32	32	32	66
Max. data rate (MB/s) ^a	8	32	40	132	132	508
Plug and play capable	No	No	Yes	Yes	Yes	Yes

^aMB/s megabytes/second

Fig. 6.14 PCI card

6.9.6 Universal Serial BUS (USB)

The *universal serial bus (USB)* is a computer serial bus which enables users to connect peripherals such as the mouse, keyboard, modem, CD-ROM, scanner, and printer, to the outside of a computer without any configuration. Personal computers equipped with USB will allow the user to connect peripherals to the computer, and the computer will automatically be configured as the devices are attached to it. This means that a USB has the capability to detect when a device has been added or removed from a PC. USB is a true plug-and-play bus. Up to 127 peripherals can be connected to a PC with a USB. USB version 1.1 was released in 1998 which supports data rate of 12 Mbps (full speed) and 1.5 Mbps (low speed); the low speed is used for devices such mouse, keyboards, and joysticks. The USB version 2 is a high speed (480 Mbps) that is compatible with USB 1.1 The USB 2.0 specification was developed by seven leading computer manufacturers and it was announced in 1999. The maximum cable length for USB is 5 m.

6.9.7 USB Architecture

Figure 6.15 shows the USB architecture; the USB system is logically a tree topology but physically is a star topology because each USB device communicates directly with the Root Hub. There is only one host controller in any USB system.

A USB system consists of USB host controller, USB root hub, USB hub, USB cable, USB device, client software, and host controller software.

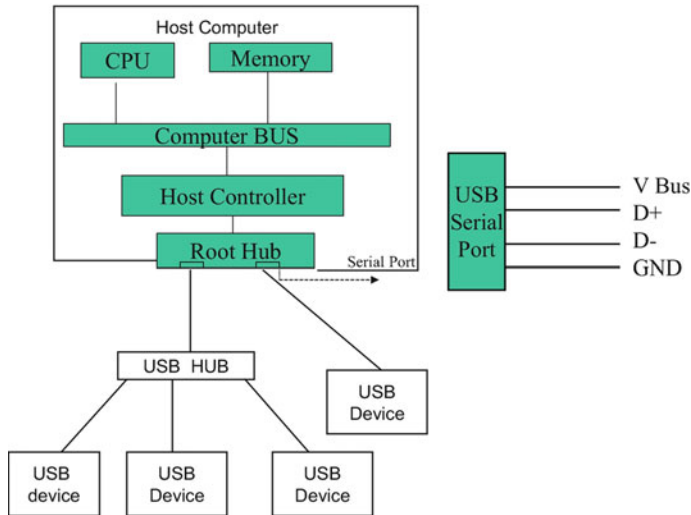


Fig. 6.15 Architecture of USB

6.9.7.1 Host Controller

The host controller initiates all data transfer, and root hub provides a connection between devices and host controller. The root hub receives transaction generated by host controller and transmits to the USB devices. The host controller uses polling to detect a new device and is connected to the bus or disconnected from. Also, USB host controller performs the following functions:

- (a) Host controller sets up the device for operation (device configuration).
- (b) Packet generation.
- (c) Serializer/deserializer.
- (d) Process request from device and host.
- (e) Manage USB protocol.
- (f) Managing flow between host and USB devices.
- (g) Assign address to the devices.
- (h) Execute client software.
- (i) Collecting status bit from USB ports.

6.9.7.2 Root Hub

The root hub performs power distribution to the devices, enables and disables the ports, and reports status of each port to the host controller. The root hub provides the connection between the host controller and USB ports.

6.9.7.3 Hub

Hubs are used to expand the number of devices connected to the USB system. Hubs are able to detect when a device is attached or removed from port. Figure 6.16 shows the architecture of hub. The upstream port is connected to the host, and USB devices are connected to downstream port.

In downstream transmission, all devices that are connected to the hub will receive the packet, but only the device accepts the packet that the device address matches with address in the token. In upstream transmission, the device sends the packet to the hub, and hub transmits the packet to its upstream port only. USB improves its speed and every few years a new version was developed. Table 6.4 shows different versions of USB.

6.9.7.4 USB Cable

Figure 6.15 shows USB port with four pins, which consists of four wires, with the V bus used to power the devices. D+ and D– are used for signal transmission.

6.9.7.5 USB Device

USB device is divided into the classes such as hub, printer, or mass storage. The USB device has information about its configuration such as class, type, manufacture ID, and data rate. Host controller uses this information to load device software from the hard disk. USB device might have multiple functions such as a volume in a speaker. Each function in a USB device is specified by the endpoint address.

Fig. 6.16 Architecture of hub

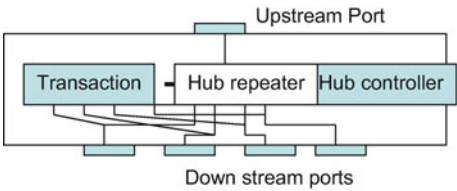


Table 6.4 USB version and its data rate

USB version	Release data	Data rate	Data rate designation
USB 1.0	1996	1.5 Mbps	Low speed
USB 1.1	1998	12 Mbps	Full speed
USB 2.0	2000	480 Mbps	High speed
USB 3.0	2008	5 Gbps	Super speed
USB 3.1	2013	10 Gbps	Super speed +

6.9.8 PCI Express Bus

PCI express was introduced in mid-1990 with 33 MHz frequency, and during the time the speed of BUS was increased to 66 MHz. Due to new development in networking technology such as Gigabit Ethernet and I/O devices that demand more bandwidth, there is a need for a new bus technology with higher bandwidth. The PCI express was approved by Special Interest Group in 2002, and chipset starts shipping in 2004. The PCI express has the following features:

- PCI express is point-to-point connection between devices.
- PCI express is a serial bus.
- PCI express uses pocket and layer architecture.
- Compatible with PCI bus through software.
- End-to-end link data integrity (error detection).
- Isochronous data transfer.
- Selectable bandwidth.

6.9.8.1 PCI Express Architecture

Figure 6.17 shows PCI express architecture, and the function of host bridge is to interface CPU bus with memory and PCI express switch. The switch is used to increase the number of PCI's express ports.

6.9.8.2 PCI Express Protocol Architecture

Figure 6.18 shows PCI express protocol architecture. The protocol consists of PCI software, transaction, data link, and physical layer.

Fig. 6.17 PCI express architecture

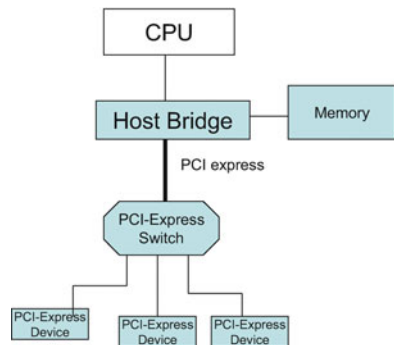


Fig. 6.18 PCI express protocol architecture

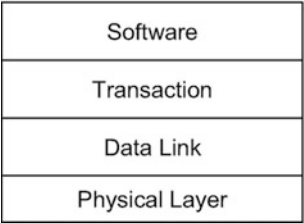
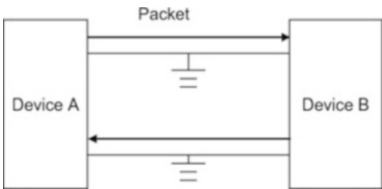


Fig. 6.19 PCI express connections



6.9.8.3 Software Layer

The software layer is used for compatibility with PCI, initialization, and enumeration of the devices connected to the PCI express.

6.9.8.4 PCI Express Physical Layer

Figure 6.19 shows two devices which are connected through PCI express link (lane); each lane is made of four wires, and each PCI express lane consists of two simplex connections, one for transmitting the packet and another one for receiving the packet. The PCI express lane supports 2.5 giga transfer/s each direction.

PCI express link may configure X1, X2, X4, X4, X16, and X32 lane, where X1 means 1 lane with 4 wires and X4 means 4 lanes with 16 wires and finally X32 means 32 lanes with 128 wires. PCI-32 means PCI express with 32 lanes. The clock for PCI express serial link is embedded into the data by using 8B/10B encoding.

6.10 FireWire

FireWire or IEEE 1394 is a high-speed serial bus used for connecting digital devices such as a digital video or camcorder. The bus is able to transfer data at the rate of 100, 200, or 400 Mbps. The IEEE 1394 cable consists of six copper wires; two of the wires carry power and four of the wires carry signal as illustrated in Table 6.5. Some FireWire connectors come with four pins, without having power pins. Figure 6.20 shows FireWire male and female connectors.

Table 6.5 IEEE 1394 pins

Pin	Signal name	Description
1	Power	Unregulated DC; 17–24 V no load
2	Ground	Ground return for power and inner cable shield
3	TPB–	Twisted-pair B, differential signals
4	TPB+	Twisted-pair B, differential signals
5	TPA–	Twisted-pair A, differential signals
6	TPA+	Twisted-pair A, differential signals

Fig. 6.20 FireWire male and female connectors

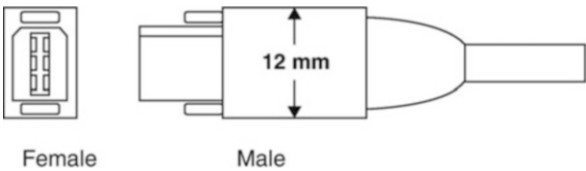


Fig. 6.21 HDMI connectors

6.10.1 HDMI (High-Definition Multimedia Interface)

HDMI is an interface between two devices for transferring uncompressed video data and compressed or uncompressed digital audio data. Some of the applications of HDMI are computer monitor, digital TV, and video projector. Figure 6.21 shows different types of HDMI connectors.

6.10.1.1 Motherboard

The motherboard is a printed circuit board (PCB) that contains most components of a computer such as CPU, RAM, ROM, expansion slots, and USB. Figure 6.22 shows an image of motherboard.

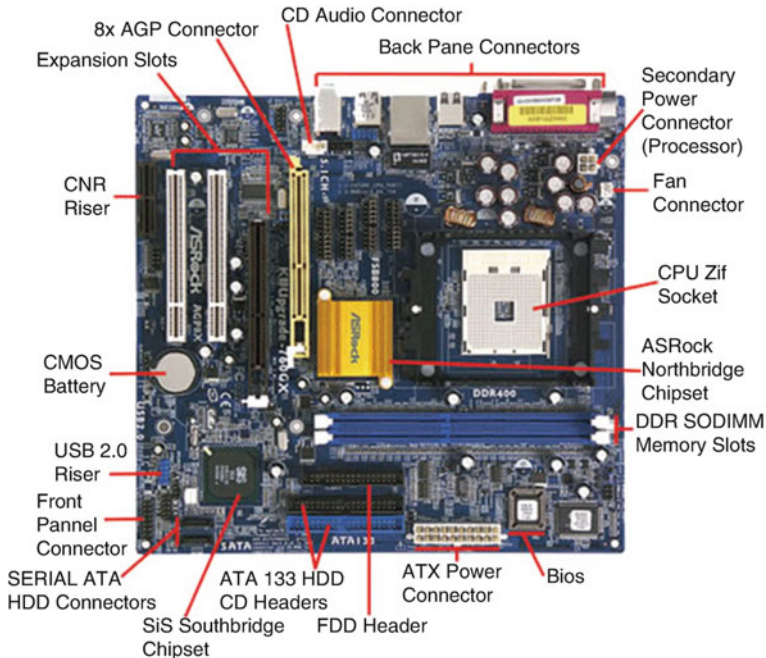


Fig. 6.22 Image of motherboard

6.11 Summary

- The components of a microcomputer are the CPU, memory, parallel I/O, serial I/O, programmable interrupt, and DMA.
- The function of the CPU is executing instruction.
- The components of the CPU are the arithmetic logic unit (ALU), control unit, and registers.
- Most computers use three types of memory: cache memory (SRAM), main memory (DRAM or SDRAM), and secondary memory (hard disk, tape drive, and floppy disk).
- Semiconductor memory types are DRAM, SDRAM, EDORAM, DDR, SDRAM, RDRAM, ROM, and EPROM.
- SRAM is used in the cache memory; DRAM and SDRAM are used in the main memory.
- SATA, SCSI-1, SCSI-2, and SCSI-3 are the computer peripheral controllers.
- ISA bus, EISA, MCA, and PCI are the microcomputer buses, and FireWire is a high-speed serial bus with a data rate up to 400 Mbps.
- USB is a serial bus.
- PCI express is a serial bus.
- Chapter 7 covers semiconductor memory, hard disk, solid-state drive, cache memory mapping methods, and virtual memory.

Review Questions

- *Multiple choice questions*

1. The function of the _____ is to perform arithmetic operations.
 - (a) Bus
 - (b) Serial port
 - (c) ALU
 - (d) Control unit
2. When you compare the functions of a CPU and a microprocessor, _____.
 - (a) They are the same
 - (b) They are not the same
 - (c) The CPU is faster than microprocessor
 - (d) The microprocessor is faster than CPU
3. RISC processors use _____.
 - (a) Complex instruction sets
 - (b) Reduced instruction sets
 - (c) (a) and (b)
 - (d) None of the above
4. The CISC processor control unit is _____.
 - (a) Hardware
 - (b) Microcode
 - (c) (a) and (b)
 - (d) None of the above
5. Direct memory access allows for the transfer of blocks of data from memory to an I/O device (or vice versa) without using the _____.
 - (a) CPU
 - (b) Data bus
 - (c) Control bus
 - (d) DMA controller
6. Which of the following buses are 32-bit?
 - (a) ISA
 - (b) PCI and EISA
 - (c) EISA and ISA
 - (d) MCA and ISA

7. How many Memory location does have a memory with 12 Address Lines

- (a) 1024
- (b) 2048
- (c) 4096
- (d) 256

8. How many Memory location does have a memory with 16 Address Lines

- (a) 1K
- (b) 4K
- (c) 64K
- (d) 32K

9. How many Memory location does have a memory with 22 Address Lines

- (a) 10K
- (b) 1M
- (c) 2M
- (d) 4M

10. The Fetch instruction means

- (a) Executing Instruction
- (b) Read Instruction from memory
- (c) Decode Instruction
- (d) Store Data

- Short Answer Questions

1. Show Abstract of a Computer?
2. What is function of OS?
3. What is function Compiler?
4. What is function of Assembler?
5. What is application of Firmware?
6. List the components of a microcomputer.
7. Explain the functions of a CPU.
8. List the functions of an ALU.
9. What is the function of a control unit?
10. List components of a CPU?
11. How many bits a half word?
12. How many bits is a word?
13. Explain the function of an address bus and a data bus.
14. Explain the function of DMA.
15. What is the application of a parallel port?
16. What is the application of a serial port?
17. What is maximum memory for a CPU with 16 address lines and 8 data lines?
18. List the types of memory use in a computer.
19. What is the type of memory use for cache memory?

20. What is the type of memory use for main memory?
21. What are the type of memory use for secondary memory?
22. What are the characteristics of a 32-bit machine?
23. What are the characteristics of a 64-bit machine?
24. List characteristics of CISC processor.
25. List characteristics of RISC processor.
26. Distinguish between von Neumann architecture and Harvard architecture.
27. What is the advantage of multicore processor versus single core?
28. List CPU instruction execution steps.
29. Explain fetch instruction
30. Explain the decode instruction.
31. What does the PC (register) stand for, and what is its function?
32. What does IR stand for?
33. How long does it take for a CPU to execute five instructions using pipelining if each stage of pipeline takes 20 ns.
34. Calculate execution time for question#33 using non-pipeline processor.
35. List types of disk controller.
36. List two computer buses.
37. List two serial buses.
38. What is maximum number USB ports a computer can have?
39. Show pin connection of USB port.
40. Show diagram of PCIe lane.
41. What is the application of FireWire?
42. What is application of HDMI?

Chapter 7

Memory



Objectives: After Completing this Chapter, you Should Be Able to

- Distinguish different types of semiconductor memory.
- Explain sector, track on hard disk.
- To calculate disk capacity.
- Learn memory hierarchy.
- Explain cache miss, cache hit, and cache hit ratio.
- Describe types of memory use in a computer.
- Explain different mapping methods use in cache memory.
- Translate virtual address to physical address.
- Explain function of page table.
- Generate physical address from virtual address.

7.1 Introduction

In a computer, memory holds instructions (code) and data, memory plays an important part of a computer performance, and register is a type of memory with small capacity. There are two types of memory used in a computer, and they are classified as semiconductor memory and hard disk. Semiconductor memory can be volatile or non-volatile memory. Volatile memory loses its contents when power is removed from it, while non-volatile memory will keep its contents without power.

7.2 Memory

Computer memory can be classified as volatile and non-volatile memory.

Volatile Memory: It requires power in order to hold information and they are Random Access Memory (RAM) and Static Random-Access Memory

(SRAM). Volatile memory is used for temporary storage and it is faster than non-volatile memory.

Non-Volatile Memory: Non-volatile memory does not require power to hold information and they are used for long time storage such as Read-Only Memory (ROM), Non-Volatile Random-Access Memory (NVRAM) such as Flash Drive, Hard Disk, and Solid-State Drive (SSD).

7.2.1 RAM

Data can be read from or written into *random-access memory (RAM)*. The RAM can hold the data as long as power is supplied to it and it is called *volatile memory*. Figure 7.1 shows a general block diagram of RAM consisting of a data bus, address bus, and read/write signals. The data bus carries data out from or into the RAM. The address bus is used to select a memory location. The read signal becomes active when reading data from RAM, and the write line becomes active when writing to the RAM. Remember, RAM can only hold information when it has power. Figure 7.2 shows a $16 * 8$ bit RAM or $2^4 * 8$ bit or 16 B RAM.

In Fig. 7.2, the address is 4 bits; therefore, there are $2^4 = 16$ memory locations, and if each location holds 1 B, then there is 16 B of memory, a memory with m address lines, then there is 2^m memory locations. Table 7.1 shows the number of address lines and equivalent decimal number of memory locations.

There are many types of RAM such as *dynamic RAM (DRAM)*, *synchronous DRAM (SDRAM)*, *EDO RAM*, *DDR SDRAM*, *RDRAM*, and *static RAM (SRAM)*.

- *Dynamic RAM (DRAM)* is used in main memory. DRAM uses fewer components to make one bit; therefore, it can design DRAM integrated circuit (IC) with large capacity as 4 GB per IC; Fig. 7.3 shows one bit DRAM.

The cell capacitor can be charged with logic one or zero, but it requires to be refreshed (recharged) about every 1 ms. The CPU cannot read from or write to memory while the DRAM is being refreshed; this makes DRAM the slowest running memory.

Fig. 7.1 RAM block diagram

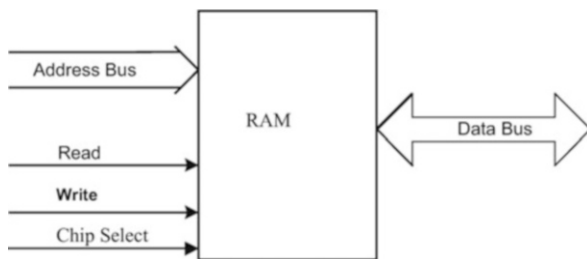


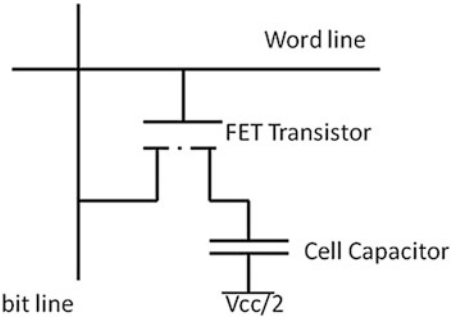
Fig. 7.2 16 bytes of RAM

Address	
0000	10101011
0001	11001100
0010	10000001
0011	10000000
0100	11111111
0101	11111000
0110	10000000
0111	00000000
1000	00111111
1001	00000001
1010	01010101
1011	10000000
1100	00000011
1101	10000011
1110	11100000
1111	11000000

Table 7.1 Number of address and memory locations

Number of addresses	Number of memory locations	Representation
10	$2^{10} = 1024$	1 K
11	$2^{11} = 2048$	2 K
12	$2^{12} = 4096$	4 K
13	$2^{13} = 8192$	8 K
14	$2^{14} = 16,384$	16 K
16	$2^{16} = 65,536$	64 K
20	$2^{20} = 1,048,576$	1 M
24	$2^{24} = 16,777,261$	16 M
32	$2^{32} = 4,294,967,296$	4 G

Fig. 7.3 1-bit DRAM



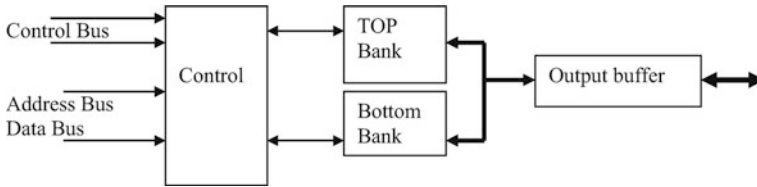


Fig. 7.4 Block diagram of SDRAM

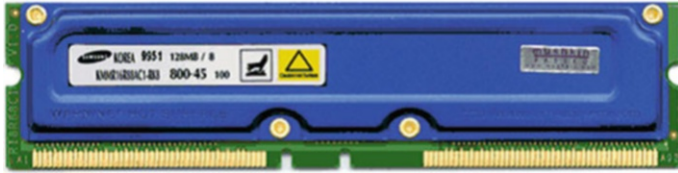


Fig. 7.5 Rambus memory module (Courtesy Samsung Corp.)

- *Synchronous DRAM (SDRAM)*: SDRAM technology uses DRAM and adds a special interface for synchronization. It can run at much higher clock speeds than DRAM. SDRAM uses two independent memory banks. While one bank is recharging, the CPU can read and write to the other bank. Figure 7.4 shows a block diagram of SDRAM.
- *Extended Data Out RAM (EDORAM)* transfers blocks of data to or from memory.
- *Double Data Rate SDRAM (DDR SDRAM)* is a type of SDRAM that transfers data at both the rising edge and the falling edge of the clock. It can move data twice faster than SDRAM; therefore, memory can run at the $\frac{1}{2}$ clock rate. DDR2 and DDR3 are an advancement on the DDR technology and further increase the number of data transfers per clock cycle. DDR2 RAM provides 4 data transfers per cycle, and DDR3 transfers 8 data per clock cycle. For 100 MHz clock rate and 64 bits data bus, the transfer rates for DDR are

$$\text{DDR} = 100 \times 2 \times 8 = 1600 \text{ MB/s (MB/s).}$$

$$\text{DDR2} = 100 \times 4 \times 8 = 3200 \text{ MB/s.}$$

$$\text{DDR3} = 100 \times 8 \times 8 = 6400 \text{ MB/s.}$$

- *Rambus DRAM (RDRAM)* was developed by Rambus Corporation. It uses multiple DRAM banks with a new interface that enables DRAM banks to transfer multiple words and also transfers data at the rising edge and the falling edge of clock. The RDRAM refreshing is done by the interface. The second generation of RDRAM is called DRDRAM (Direct RDRAM), and it can transfer data at a rate of 1.6 Gbps. Figure 7.5 shows a RDRAM module.

7.2.2 DRAM Packaging

DRAM comes in different types of packaging such as SIMMs (single in-line memory module) and DIMMs (dual in-line memory module).

Figure 7.6 shows SIMM, which is a small circuit board that one side of the board holds several chips. It has a 32 bit data bus.

DIMM is a circuit board that both sides of the board hold several memory chips but has a 64 bit data bus.

- *Static RAM (SRAM)* is used in cache memory. SRAM is almost 20 times faster than DRAM and is also much more expensive. Figure 7.7 shows diagram of 1 bit SRAM which is used for 6 MOSFET transistors.

7.2.3 ROM (Read-Only Memory)

Like its name suggests, information can be read only from *read-only memory (ROM)*. ROM holds information permanently, even while there is no power to the ROM; this type of memory is called *non-volatile memory*. Two types of ROM are listed below:

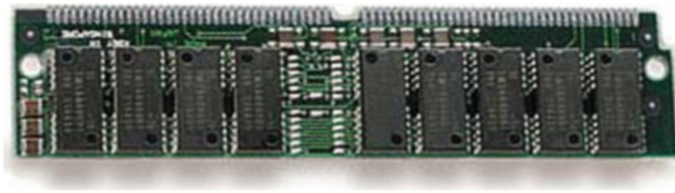


Fig. 7.6 DRAM SIMM

Fig. 7.7 1 bit SRAM

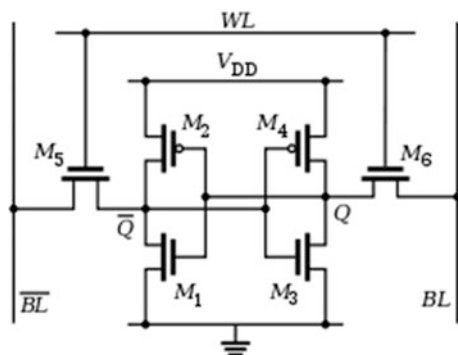


Table 7.2 Memory access time

Memory technology	Access time
SRAM	0.5–2.5 ns
DRAM	50–70 ns
Flash	$5 * 10^3 - 5 * 10^5$

- *Erasable Programmable Read-Only Memory (EPROM)*: EPROM can be erased with ultraviolet light and reprogrammed with a device called an EPROM programmer. Flash ROM is a type of EEPROM.
- *Electrically Erasable PROM (EEPROM)*: EEPROM can be erased by applying specific voltage to one of its pin and can be reprogrammed with an EPROM programmer.
- *Flash Memory*: flash memory is a non-volatile memory that has wide range of applications such as flash drive, solid-state drive, memory card, and embedded system. Flash memory is a type of EEPROM that allows multiple memory location to be written or erased on one operation. There are two types of technology use for flash memory, and they are NAND and NOR flash memories; NAND flash memory has smaller access time than NOR flash memory; most flash memory uses NAND technology.

7.2.4 Memory Access Time

The time the CPU places address on address bus and data appears on data bus or write the data into memory. Table 7.2 shows access time for different types of memory.

7.3 Hard Disk

Figure 7.8 shows the internal architecture of hard disk made of several platters, and platters hold the information; the functions of the heads are to read or write information from disk surface. The surface of platter is made of several tracks, and each track is divided into sectors as shown in Fig. 7.9.

7.3.1 Disk Characteristics

Access Time: the time that takes to start transfer data, and it is the sum of seek time and rotational delay.

Seek Time: the time that takes the head move to the proper track.

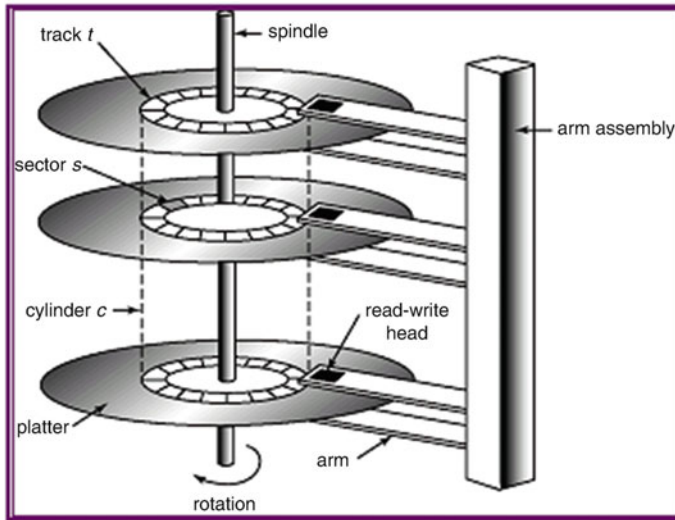


Fig. 7.8 Internal architecture of hard disk

A : Track
B : Geometrical Sector
C: Track Sector
D: Cluster

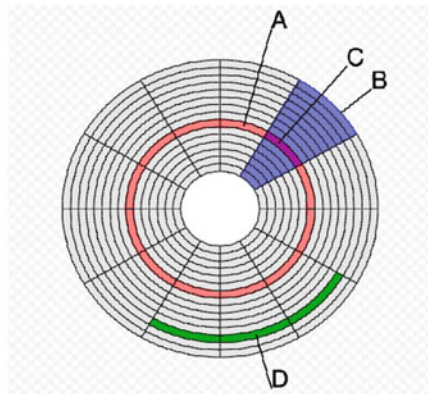


Fig. 7.9 Surface of a platter

Rotational Delay: the time that it takes a sector to be positioned under read/write head and depend on rotation speed. The rotation speed represented by revolutions per minute (RPM) assumes the sector is away from head half of the track; therefore the rotation delay is calculated by

$$\text{Rotational delay} = \text{Time for half revolution} = 60 \text{ s/RPM} * 2$$

Disk Capacity: capacity of a disk calculated by

Table 7.3 Default cluster size

Disk size	NTFS cluster size
512–1024 MB	1 kB
1024 MB–2 GB	2 kB
2 GB–2 TB	4 kB

$$\text{Disk capacity} = \text{Number of surfaces} * \text{Number of track/Surface} * \text{Number of sectors/Track} * \text{Number of bytes/Sector}$$

7.3.2 Cluster

Each sector of a disk is 512 bytes (B), and cluster is made of one or more sectors, if a cluster is 1 kB, then it is made of two sectors. The 2 kB cluster is made of 4 sectors. Table 7.3 shows default values of cluster size.

Example 7.1 A disk drive has 8 surfaces, each surface has 1024 tracks, each track has 64 sectors, and each sector can hold 512 B and rotation speed of 6000RPM.

- (a) What is the capacity of disk?
- (b) What is the rotational delay?

$$\text{Disk capacity} = 8 * 1024 * 64 * 512 = 268,435,456 \text{ B.}$$
$$\text{Rotation delay} = 60/6000 * 2 = 0.005 \text{ s.}$$

7.3.3 Disk File System

A file system defines organization of information stored in hard disk; the windows OS (operating system) offers FAT16 (file allocation table) and FAT32 which are used for early windows OS; currently most windows OS uses NTFS (New Technology File System). NTFS offers better security such as permission to restrict access and encryption.

7.4 Solid-State Drive (SSD)

Hard drive is a slow device, and it can be replaced by SSD. SSD is made of non-volatile NAND flash memory. Figure 7.10 shows architecture of SSD, and Table 7.4 compares SSD with HDD.

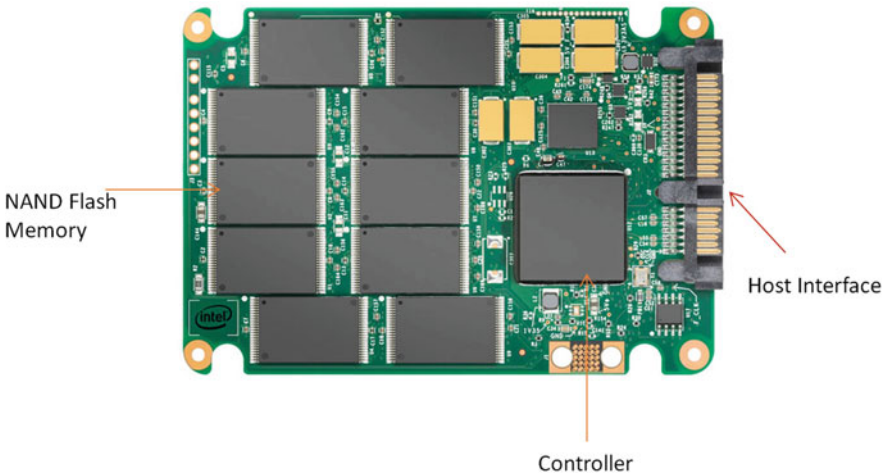
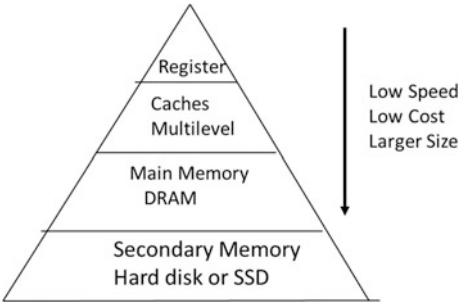


Fig. 7.10 Architecture of SSD

Table 7.4 Comparing the SSD with HDD

Characteristics	SSD	HDD
Access time	100 times faster than HDD	5000–10,000 μ s
Price	Expensive	Less expensive
Reliability	More reliable because it does not have any mechanical part	Less reliable
Capacity	Gigabytes	Terabytes
Power	Less power than HDD	More power than SSD

Fig. 7.11 Memory hierarchy of a microcomputer



7.5 Memory Hierarchy

Computers come with four types of memory, which are arranged in a hierarchical fashion, as shown in Fig. 7.11:

Table 7.5 Show price of different type of memory

Memory type	SRAM	DRAM	SSD	HDD
Cost	\$8.00/MB	\$0.16/MB	\$0.20/GB	0.05/GB
Access time	0.5–2.5 ns	50–70 ns	70–150 ns	5–20 ms

1. *Register*: Register is fastest memory and it can read and write to it by single clock cycle.
2. *Cache memory*: Cache memory is the fastest type of memory and is most often used inside CPU called L1 cache, and it is faster than main memory and, therefore, more expensive than main memory.
3. *Main memory*: Main memory uses DRAM and SDRAM. The program to be executed moves from secondary memory (disk or tape) into main memory.
4. *Secondary memory*: Second memory refers to memory such as hard disk, SSD, and CD-ROM (Table 7.5).

7.5.1 Cache Memory

Each cache memory location is called cache line which can hold a block of data from main memory. For the most processors, the cache memory is located inside CPU and called L1 cache; there are two types of cache in a CPU:

1. Data cache (D-cache): Data cache holds the data and it can be read or write by CPU.
2. Instruction cache (I-Cache): Instruction cache holds instruction and CPU only read from I-cache.

7.5.2 Cache Terminology

Miss: when CPU accesses the cache and data is not in cache, it is called cache miss

Hit: when CPU accesses the cache and data is in the cache, then it is called cache hit

Hit ratio: number of hits/number of miss + number of hits (total number of reads)

Block: multiple of main memory locations is called block

Physical address: address generated by CPU to access main memory

Virtual address: address generated by CPU to access virtual memory or secondary memory

Cache line or cache block: each can line or block holds multiple bytes or words; the size of cache line is the same block in main memory

Temporal locality: once a memory location referenced, then there is a high probability to be referenced again in near future

Spatial locality: when a memory location is accessed, then it is very likely the nearby locations will be accessed soon.

7.5.3 Cache Memory Mapping Methods

Figure 7.12 shows a cache with 4 locations and main memory with 8 memory locations, and the cache can hold only 4 memory locations of main memory; CPU first accesses the cache if data is not in cache and then accesses main memory and moves a block data into the cache; the question is where the data will store in the cache; this bring subject of mapping methods.

The mapping methods are used to map a block of main memory into the cache line (cache block), and the following are types of methods used for mapping:

1. Direct mapping
2. Associative mapping
3. Set associative

7.5.4 Direct Mapping

Figure 7.13 shows a cache memory with four cache lines, and each line holds 4 B; the physical address seen by cache is divided into three fields and they are:

Offset: determines the size of cache line (number of bytes or words) since each cache line can hold 4 memory locations and then the offset is 2 bits, and each block is 4 memory locations; also offset determines which of the four data to be transferred to CPU.

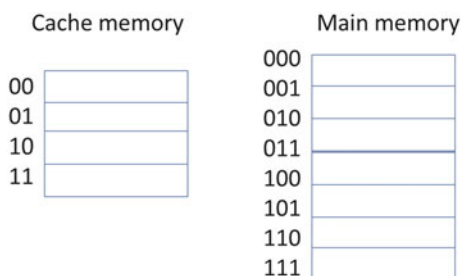
Index: index is the address to the cache; in this figure there are 4 cache lines and then index is 2 bits.

V-bit (valid bit): V-bit set to one to represent that data in cache line is valid.

Tag: $\text{Size of Tag} = \text{Size of Physical Address} - (\text{Size of Index} + \text{Size of Offset})$.

If physical address is 7 bits, then $\text{tag} = 7 - (2 + 2) = 3$ bits.

Fig. 7.12 Cache and main memory



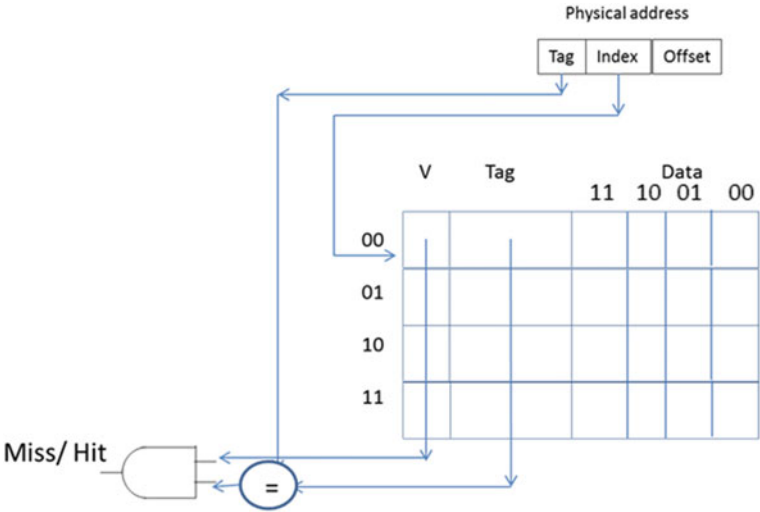


Fig. 7.13 Cache memory with 4 cache lines

Tag	Index	Offset
010	10	11

Fig. 7.14 Format of physical address for 0101011

If valid bit is one and tag in address field matches with the tag stored in cache, then results are hit; otherwise is miss.

When CPU receives miss from cache, then access the main memory and transfer a block of data from main memory to cache line using the following equation for direct mapping:

$$\text{Cache line address} = (\text{Main Memory Block Number}) \text{Modulo } N$$

where N is the number of cache lines.

Consider Fig. 7.14 cache and assume CPU generates address 010 10 11, the Fig. 7.13 shows format of physical address seen by cache.

The CPU uses the index value to access cache line 10 and if the valid bit is zero results in a miss, the CPU accesses the main memory and transfers block 01010 to cache line 10 according to the following equation:

$$(01010)_2 = (10)_{10}$$

$$\text{Cache line number} = 10 \text{ modulo } 4 = 2 \text{ or } (10)_2$$

In this case each block is made up of 4 memory locations. Since the offset is two bits, it can refer any of 4 (2^2) chunks of data within a block of memory. The block at

Fig. 7.15 Cache memory with four lines (four blocks)

	V	Tag	Data			
			11	10	01	00
00	0					
01	0					
10	1	010	M(2B)	M(2A)	M(29)	M(28)
11	0					

address *01010* in main memory with a 2 bit offset consists of memory locations 0101000 (M28), 0101001 (M29), 0101010 (M2A), and 0101011 (M2B).

In this case, M28, M29, M2A, and M2B are transferred to cache line *10*, and the V-bit is set to one. The tag is stored in the tag field of cache as shown in Fig. 7.15.

Now, if the CPU generates address *0101010*, where tag = *010*, index = *10*, and offset = *10*, the CPU uses the index to access cache line *10*. If on cache line *10* the V-bit equals *1* and the tag of the address matches with the tag in the cache line, then the result is a *hit*, and the CPU uses offset *10* to move data (M2A) in to the CPU. (Where offset *00* = M28, *01* = M29, *10* = M2A, *11* = M2B.)

Example 7.2 Figure 7.16 shows the main memory and cache memory of a computer. CPU generates (in hex) addresses 0×0 , 0×2 , 0×5 , and 0×2 . Assuming the cache is empty at the beginning, show the contents of the cache.

In this example each block and cache line is 2 B and main memory consists of 8 blocks. Figure 7.17 shows the physical address as seen by the cache.

- The byte offset is one bit. (2 B per block).
- The index is two bits. (cache consist of 4 lines = 2^2).
- The tag is one bit. (Bits in block – bits in index = $3 - 2 = 1$).
- The CPU generates address 0×0 or *0000*. Therefore, the tag is *0*, the index is *00*, and the offset is *0*. The CPU accesses cache line *00* and the V-bit is zero which results in a *miss*. The CPU accesses main memory address *0000*, transfers block *000* to the cache line *00*, and sets the tag bit to zero and the V-bit to one. Therefore, cache line *00* contains: V = *1*, Tag = *0*, Byte1 = 0×5 , Byte0 = 0×1 .
- Next the CPU generates address 0×2 or *0010*. The index is *01* so the CPU accesses cache line *01*, where the valid bit is zero which results in a *miss*. Then the CPU accesses main memory location *0010* and transfers block *001* into cache line *01*, changes the valid bit to one, and stores the tag part of the address into the cache line's tag. Therefore, cache line *01* is now: V = *1*, Tag = *0*, Byte1 = 0×7 , Byte0 = 0×6 .
- The CPU generates address 0×5 or *0101*. The index is *10*, so the CPU accesses cache line *10* and the result is a *miss*. The CPU accesses main memory location

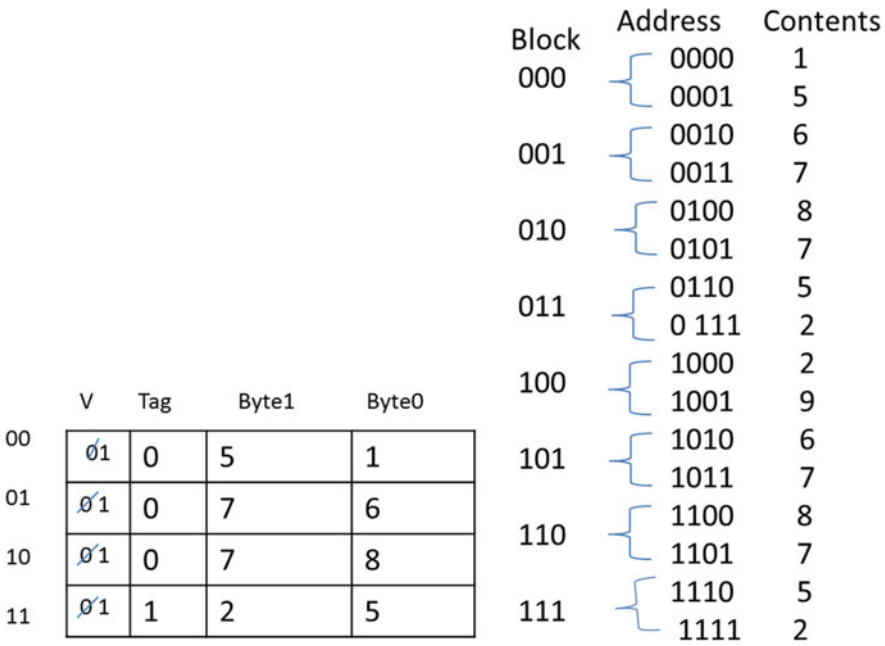
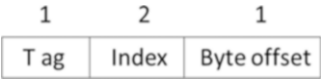


Fig. 7.16 Cache memory and main memory of Example 7.2

Fig. 7.17 Physical address format for Example 7.1



0101 and transfers block 010 to the cache, sets V to one, and stores the tag of the address into the cache’s tag. Cache line 10 is now: V = 1, Tag = 0, Byte1 = 0 × 7, Byte0 = 0 × 8.

- The CPU generates address 0 × 2 or 0010, so it accesses cache line 01. The valid bit is 1 and the cache line tag matches the address tag, which results in a hit. The offset is 0, so the CPU reads byte 0 from cache line 01. This process will continue for other addresses.

Using the same size cache and main memory as above, blocks 000 and 100 will both be mapped onto cache line 00. If the CPU generates addresses 0000, 1000, 0001, and 1001 consecutively, then results will be misses for all four addresses. In order to reduce misses, then the cache can be divided into sets, and mapping method is called set associative mapping.

7.5.5 Set Associative Mapping

In set associative mapping, the cache memory is divided into sets. The size of the set can vary: examples include two-way set associative, four-way set associative, and so on. Figure 7.18 shows a two-way set associative cache, and Fig. 7.19 shows the physical address format seen by the cache. Using the previous examples as a base, the byte offset will remain one bit since it refers to one of two bytes stored on the cache line. The *set* identifier will be one bit that defines the set address; the tag size is calculated as follows:

Tag size = Physical address size – Set address – Offset

Assume the physical address is 4 bits, then Fig. 7.19 shows the physical address seen by cache.

Example 7.3 Consider the main memory from Fig. 7.16, where the cache is empty and it is divided into two sets as shown in Fig. 7.20. Show the contents of the cache if the CPU generates addresses 0 × 0, 0 × 8, 0 × 0, and 0 × 8.

Tag Size = Physical address size – Set address – Offset

	V	Tag	Byte1	Byte0
0	0			
1	0			

	V	Tag	Byte1	Byte0
0				
0				

Fig. 7.18 Two-way set associate cache

2	1	1
Tag	Set	Byte Offset

Fig. 7.19 Physical address seen by cache for set associative mapping

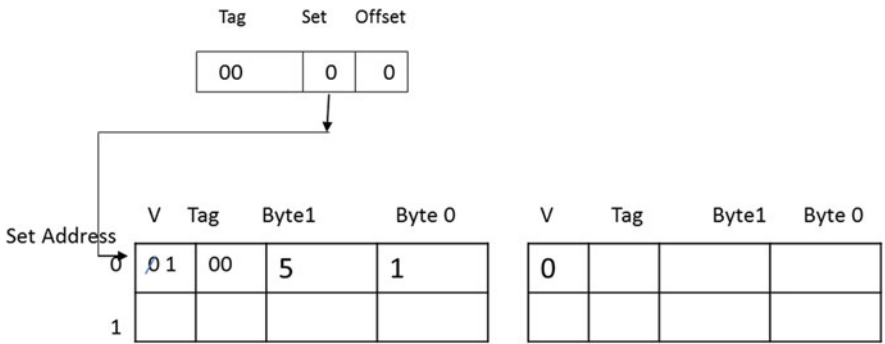


Fig. 7.20 Content of cache for address 0000

- The CPU generates address 0×0 or 0000 and accesses cache set 0 . Both cache lines in the set 0 have valid bits of 0 , so the result is a *miss*. The CPU then accesses main memory, transfers contents of memory locations 0000 and 0001 into the cache, then changes the valid bit to one, and stores the tag part of the address into the cache's tag.

Next, the CPU generates address 0×8 or 1000 and accesses cache set 0 . First line of the cache has a valid bit of 1 but the tag does not match (10 vs. 00). The CPU then accesses main memory, transfers contents of memory locations 1000 and 1001 into second line of the cache in the set 0 , and changes the valid bit to one and stores the tag part of the address into the cache's tag. Now, when the CPU generates the addresses 0×0 and 0×8 again result *hits*.

7.5.6 Replacement Method

In set associative, when CPU brings a new block into cache, then one of the cache lines must be replaced with new block; consider Fig. 7.21. If CPU generates address 0100 (4) (tag = 01 , set address = 0 , and offset is 0), it accesses set 0 , and both cache lines have valid bit one, but tags in cache lines do not match with address tag results miss, and then CPU accesses main memory and must move the contents of memory locations 0100 and 0101 into cache. CPU uses *least recently used (LRU)* method which moves new block from main memory and replaces it with block that has been longer in the cache. This method accomplishes by adding LRU bit to each cache line of the cache as shown in Fig. 7.22.

In Fig. 7.22, assume both cache lines in set 0 are empty, a new block moves into the first cache line in set 0 , and LRU changed from 0 to 1 ; the second block moves to second cache line of set 0 , and LRU changed from 0 to 1 , but at the same time, LRU of first cache line will change from 1 to 0 ; therefore, the cache line with $LRU = 0$ contains block that has been longer in cache line.

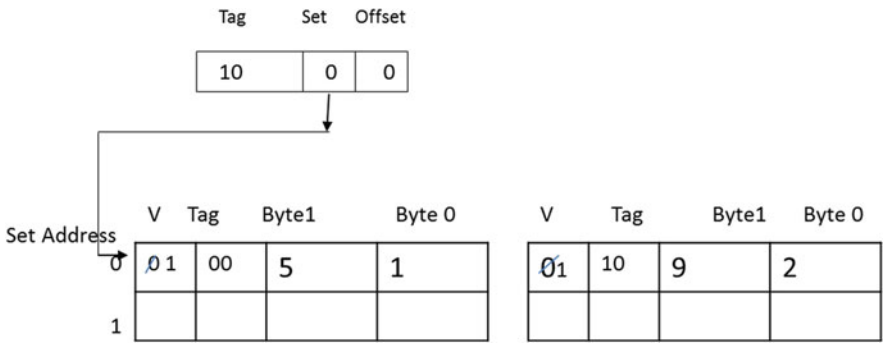


Fig. 7.21 Contents of cache for addresses 0000 and 1000

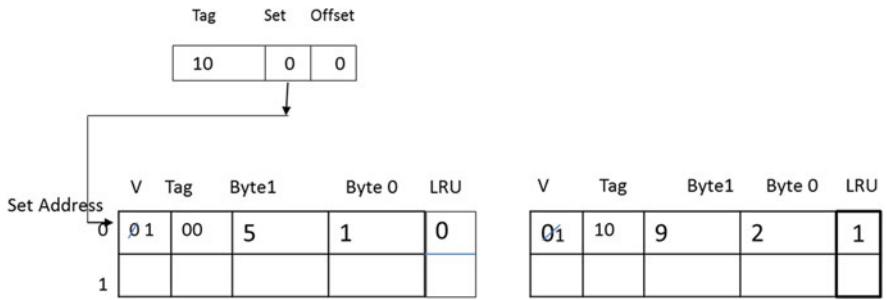


Fig. 7.22 Two set associative with LUR

Fig. 7.23 Associative mapping

Valid	Address	Byte1	Byte0
1	000	5	1
1	100	9	2
0			
0			

7.5.7 Fully Associative Mapping

In fully associative mapping, the entire address is stored in the cache with its data. Figure 7.23 shows a fully associative cache with four lines after the CPU has accessed main memory locations 0×0 and 0×8 . If the CPU next generates address 0×0 or 0000, then it will compare the address with each address in the cache, and if it matches, then it will read the data from the cache.

7.5.8 Cache Update Methods

- 1. *Write Through*: When new information is written to the cache, main memory is also updated.
- 2. *Buffered Write Through*: There is a buffer between cache and main memory, when new information is written to the cache, this information is written into buffer, and the CPU can access this memory before the new information can be written into main memory.
- 3. *Write Back (Copy Back)*: In this method, only the cache is updated, and main memory will be updated when the corresponding cache line is overwritten. In this method each cache line has a *dirty bit* to indicate if cache line has been modified or not.

7.5.9 Effective Access Time (EAT) of Memory

The performance of memory depends on hit ratio of cache, the EAT of memory is calculated by

$$\text{EAT} = H * T_c + (1 - H) * T_m$$

The H is hit ratio of the cache, T_c is the cache access time, $(1-H)$ is miss rate, and T_m is memory access time.

Example 7.4 A computer has a cache memory with access 10 ns and main memory access time 100n, a program was executed and result 90% hit ration. What is EAT of the memory

$$\text{EAT} = 0.9 * 10 + (1 - 0.9)200 = 29 \text{ ns}$$

7.5.10 Virtual Memory

Virtual memory is a HDD or SSD; it is used to store application data and instruction that is currently not needed to be process by the CPU. Virtual memory enables a system to run application larger than main memory. Disk is seen by CPU as virtual memory, if CPU has 16 address lines, then the size of virtual memory will be 2^{16} B. Application resides in disk and it is called process. When user runs a program, the operating system moves the pages of process into main memory. Virtual memory is divided into the pages as shown in Fig. 7.24; the process A occupied pages P0 through P4, and process B occupied pages P5–P9.

The CPU generates a virtual address (to access an address in the disk) of V -bits. These bits are divided into two identifiers: a virtual page number of M bits and a page offset of N bits as shown in Fig. 7.25. The total number of pages in the system is equal to 2^M , and the number of bytes (in a byte-addressable system) in a page is equal to 2^N . The total number of addresses in a system is equal to $2^{(N + M)}$ or 2^V .

Example 7.4 The capacity of a virtual disk is 2 MB (megabytes), and each page is 2 kB (kilobytes) in a byte-addressable system.

- What are values of N and M ?
- How many pages are in the disk?

Since each page is 2 kB, $2^N = 2048 \text{ B}$, meaning N equals 11.

The capacity of the disk is 2 MB. $2^V = 2 \text{ M}$ equals 2^{21} so $V = 21$ bits.

The number of pages, then, is equal to $2^{(21-11)}$. The disk contains 2^{10} or 1024 pages, and the size of each page is 2 kB.

Fig. 7.24 Virtual memory

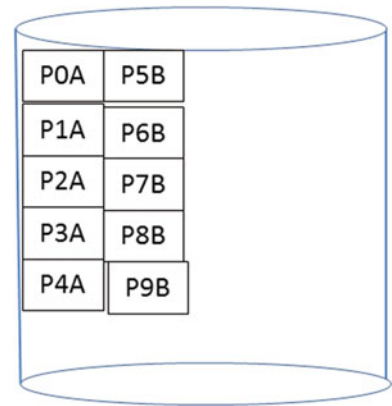
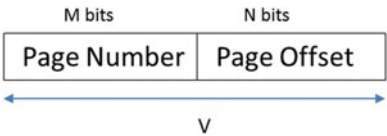


Fig. 7.25 Virtual address format



7.5.10.1 Page Table

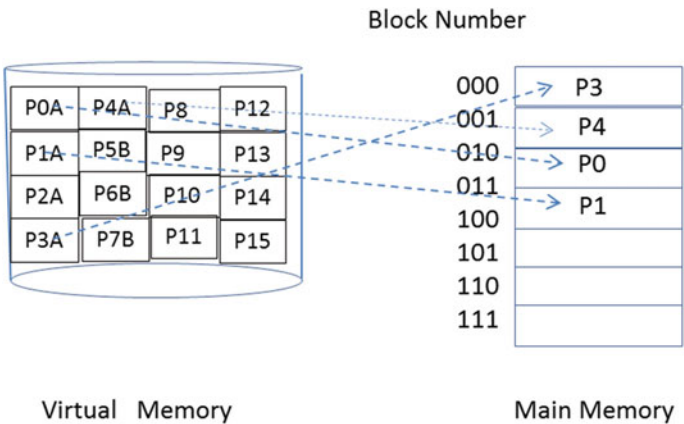
With main memory divided into blocks, the size of each block (or frame) is equal to the page size. When the CPU transfers a page into main memory, it records the page number and corresponding block in the *page table*. The address line of the page table is the page number. Each line contains the frame or block number of the matching location in main memory and a valid bit that indicates whether the line is valid or not. Figure 7.26 shows a page table wherein pages P0, P1, P3, and P4 are transferred to the blocks 2, 3, 0, and 1, respectively.

Each process has its own page table stored in main memory. Since cache memory is faster, part of the page table referred to as the *translation lookaside buffer* (TLB) is stored in the cache. The TLB uses associative mapping.

7.5.11 Memory Organization of a Computer

Figure 7.27 shows memory organization of a computer which in this example consists of:

- (a) Virtual memory (hard disk or solid-state drive).
- (b) Main memory (A type of DRAM).
- (c) Cache memory (SRAM).
- (d) A page table which keeps track of pages in main memory.
- (e) The TLB which holds a part of the page table.



	Page Number V	Block Number
0000	1	010
0001	1	011
0010	0	
0011	1	000
0100	1	001
0101	0	
0110	0	
0111	0	
1000	0	
1001	0	
1010	0	
1011	0	
1100	0	
1101	0	
1110	0	
1111	0	

Fig. 7.26 Virtual memory, memory, and page table

7.5.11.1 Memory Operation

The following steps describe the operation of memory of a computer. First, the CPU generates a virtual address and checks the TLB to see if the corresponding page is in main memory already or not.

- (a) If the TLB indicates that the corresponding page is in memory, then generate a physical address and check if the data is in the cache.
 - 1. If it is in the cache, then this called a *hit* and it reads the data from the cache.
 - 2. If it is not in the cache, then it is called a *miss*, and the CPU accesses memory and moves a block of data into the cache then reads it.

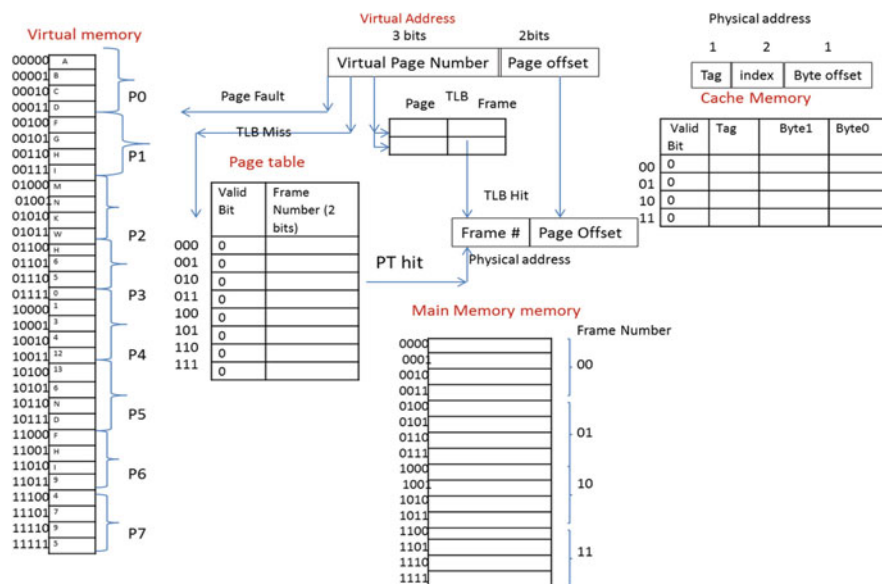


Fig. 7.27 Memory organization of a computer

- (b) If the corresponding page is not in the TLB, then the CPU checks the page table.
1. If the corresponding page is in main memory, then update the TLB and repeat from step 1.
 2. If corresponding page is *not* in main memory, then move the page from virtual memory into main memory, update the page table, update the TLB, and repeat from step 1.
- (c) If the corresponding page is not in the page table, then the CPU moves the page from virtual memory into main memory, updates the page table, updates the TLB, and repeats from step 1.

Questions and Problems

1. Distinguish between volatile and non-volatile memory.
2. What does RAM stand for?
3. List three different types of RAM.
4. Which of the following memory types are used for main memory?
 - (a) ROM and SDRAM.
 - (b) SRAM and DRAM.
 - (c) SDRAM and DRAM.
 - (d) DRAM and EPROM.

5. _____ holds information permanently, even when there is no power.
 - (a) ROM.
 - (b) DRAM.
 - (c) RAM.
 - (d) SRAM.
6. What does ROM stand for?
7. Is flash memory a type of RAM or ROM?
8. What is the difference between EEPROM and EPROM?
9. What is capacity of a memory with 10 address lines and holds one byte per memory location?
10. What is the primary application of SRAM?
11. What is the primary application of DRAM?
12. Define the following terms:
 - (a) Track.
 - (b) Sector.
 - (c) Cluster.
13. A hard disk consists of 4 surfaces, each surface consists of 80 tracks, and each track consists 32 sectors. Each sector holds 512 B. What is the capacity of this disk?
14. What is the function of file allocation table (FAT)?
15. List types of memory in a computer from fastest to slowest.
16. What are the types of cache?
17. What type of memory is used for cache memory?
18. What is virtual memory?
19. Distinguish between a virtual address and physical address.
20. Physical address determines size of
 - (a) Virtual memory
 - (b) Physical memory
 - (c) Cache memory
21. Show the format of a virtual address.
22. What is hit ratio?
23. Explain temporal locality.
24. Explain spatial locality.
25. List cache mapping methods.
26. Show format of address seen by the cache for direct mapping.
27. List cache mapping methods.
28. Show format of address seen by the cache for set associative mapping.
29. What is the function of a page number in a virtual address?
30. How many bits is the page offset if each page holds 8 kB?
31. What is the function of the page table?
32. What information is stored in TLB? Where is the TLB stored?
33. List cache mapping methods.

34. What is the advantage of set associative versus direct mapping of caches?
35. What are the three write policies used for memory?
36. _____ is the fastest type of memory.

(a) Cache memory

(b) Main memory

(c) Secondary memory

(d) Hard disk

Problems

1. The following main and cache memory are given. The CPU generates addresses $0 \times 0, 0 \times 2, 0 \times 3, 0 \times 4, 0 \times 5, 0 \times 3, 0 \times 6, 0 \times 6, 0 \times 7, 0 \times B, 0 \times D,$ and $0 \times F$. Show the contents of the cache and find the hit ratio.

ADDRESS	Contents
0000	5
0001	0
0010	1
0011	11
0100	15
0101	09
0110	16
0111	23
1000	65
1001	01
1010	8
1011	9
1100	15
1101	0
1110	2
1111	5

v	Tag	Data

2. The Ffollowing Mmemory and cache are given, if CPU generates address 0, 1, 2, 3, 8,9, 11, 13, 1F and 1E in hex

(a) Show the contents of the cache using direct mapping

(b) What is the hit ratio?

Memory

Address	Content	Address	Content
00000	5	10000	5
00001	3	10001	0
00010	11	10010	1
00011	6	10011	11
00100	7	10100	15
00101	8	10101	09
00110	9	10110	12
00111	12	10111	23
01000	0	11000	65
01001	0	11001	21
01010	8	11010	8
01011	7	11011	7
01100	9	11100	9
01101	0	11101	0
01110	2	11110	2
01111	5	11111	5

Cache Memory

	V	Tag	Data	Data
00				
01				
10				
11				

3. Using information of problem#2 with the following cache

- (a) Show the contents of the cache
- (b) What is the hit Rate?

	V	Tag	Data	Data	Data	Data
0						
1						

4. The following memory and cache memory is given. CPU generates addresses $0 \times 1, 0 \times 2, 0 \times 1, 0 \times 8, 0 \times 9, 0 \times 1C, 0 \times 1D, 0 \times 3$, and 0×4 .

- (a) Show the contents of the cache using two-way set associative mapping; assume a LRU replacement policy.
- (b) What is the hit rate?

Set	Address	V	Tag	B1	B0	LRU	V	Tag	B1	B0	LRU
00		0				0	0				0
01		0				0	0				0
10		0				0	0				0
11		0				0	0				0

Address	Content	Address	Content
00000	5	10000	5
00001	3	10001	0
00010	11	10010	1
00011	6	10011	11
00100	7	10100	15
00101	8	10101	09
00110	9	10110	12
00111	12	10111	23
01000	0	11000	65
01001	0	11001	21
01010	8	11010	8
01011	7	11011	7
01100	9	11100	9
01101	0	11101	0
01110	2	11110	2
01111	5	11111	5

5. Assume a computer has 1 M bytes main memory and a cache of 64 blocks, where each cache block contains 8 bytes
 - (a) What is the format of address seen by cache?
 - (b) How many blocks are in main memory?
 - (c) Which block does CPU access with address 0AF56
6. Assume a computer has 512 M bytes of memory with 1024 blocks of cache and each cache block holds 16 bytes
 - (a) How many blocks are in main memory?
 - (b) What is the format of address seen by cache using direct mapping?
 - (c) What is the format of address seen by cache using 2-ways set associative?
7. A computer has 24 bit physical addresses and each memory location holds one byte. This computer has 64 cache lines and each line holds 16 Bytes. Show the format of the address (tag, index, and byte offset) using
 - (a) Direct mapping
 - (b) 4-way set associative
 - (c) 8-way set associative
8. A computer has 32 kB of virtual memory and 8 kB of main memory with a page size of 512 B.
 - (a) How many bits are in the virtual address?
 - (b) How many pages are in virtual memory?

- (c) How many bits are required for the physical address?
 (d) How many frames or blocks are in main memory?
9. A computer with 256 MB of virtual memory, 4 MB of main memory, and 8 kB of cache memory. Assume a page size of 2 kB.
- (a) What is the size of a virtual address?
 (b) What is the size of a physical address?
 (c) How many pages are in virtual memory?
 (d) How many blocks are in main memory?
 (e) What is the size of the page table? (Include the number of locations and the total size of each location including all information.)
10. A Computer Has 20 Bits of Virtual memory and each Page Is 2 kB.
- (a) What is the size of virtual memory?
 (b) How many pages are in virtual memory?
11. A computer with 4 words per block has 4 K blocks of cache and 1 M blocks of main memory.
- (a) What is the size of a physical address?
 (b) Determine the size of the tag, index, and word offset of physical address using direct mapping.
 (c) Determine the size of the tag, set, and word offset of physical address using Two-way set associative mapping.
12. CPU of Fig. 7.26 generates addresses 0×00 and $0 \times 0b$; assume page 0 map into block 1 and page 2 map in block 0, show the contents of page table.
13. A computer has 64 M bytes of virtual memory and 16 M bytes of main memory, assume each page is 4 Kbytes.
- (a) What is size of virtual address?
 (b) How many bits is Physical address?
 (c) Show address field of Virtual address (Page Number and Page Offset)?
 (d) How many blocks are in main memory?
 (e) Calculate the physical address for Virtual address 005444, assume page 5 maps to block 1.
14. Use Fig. 7.27 and assume CPU generates addresses 02, 03, 1F, 1E, and 19 in Hex, assume page 0 maps into frame# 3, page 7 maps into frame #2, and page6 maps to frame# 0,
- (a) Show the content of page table, TLB, memory, and cache memory.
- | Address in Hex | TLB Hit/Miss | Page Fault (yes/No) | Physical address |
|----------------|--------------|---------------------|------------------|
| 02 | | | |
| 03 | | | |
| 1F | | | |
- cache Hit/Miss.

1E
19

15. A computer has 1 MB bytes of virtual memory, 512 k bytes of main memory, and page size of 32 k bytes. The following figure shows TLB and page table of the computer.
- (a) How many bits are in virtual address?
 - (b) How many pages are in virtual memory?
 - (c) How many bits require for physical address?
 - (d) How many frames are in main memory?
 - (e) The following virtual addresses are given in Hex, find physical addresses and identify if each address generates page fault or not
 - 38AAA
 - 47BBB
 - 19EEE

Page Table		
	V	Frame#
00000	1	1000
00001	0	
00010	0	
00011	1	1111
00100	0	
00101	0	
00110	0	
00111	1	0100
01000	1	0101

TLB	
Page #	Frame #
00111	0100
01000	0101